

Beatriz São Leandro Cosimatti

17024130

João Victor Bozola Bosi

5979703

João Vitor Vasconcelos Jacob

16987258

Laís Antunes Cayres Quintão

16880392

Trabalho de Estrutura de Dados II:
Análise e Implementação de Algoritmos

Disciplina: Estrutura de Dados II

SCC0606

Professor: Careca safado

São Carlos
14 de Junho de 2026

1 Introdução

O objetivo do trabalho desenvolvido foi estudar e implementar diferentes algoritmos de ordenação na linguagem C. Como cada algoritmo se comporta de uma maneira específica dependendo do tamanho do vetor ou de como os números estão organizados, o objetivo final do projeto é a criação de um Sistema Adaptativo. Esse sistema funciona como um analisador que avalia o vetor de entrada e, através de heurísticas, escolhe automaticamente o melhor algoritmo para realizar a ordenação.

Para fundamentar as tomadas de decisão desse sistema, o projeto foi estruturado de maneira modular, englobando a implementação de cinco algoritmos: Bubble Sort, Insertion Sort, Quick Sort, Heap Sort e Radix Sort.

ACHO Q COLOCA O LINK DO VIDEO AQ

2 Algoritmos Base

2.1 Bubble Sort

O Bubble Sort (Ordenação por Bolha) é um algoritmo de ordenação baseado no princípio de comparações e trocas adjacentes. Ele percorre o vetor repetidamente a partir do início comparando pares de elementos vizinhos (vetor[i] e vetor [i+1]), e troca-os de lugar se o elemento da esquerda for maior que o da direita. Esse processo é repetido até que nenhuma troca seja necessária em uma passada completa, indicando que o conjunto de dados está totalmente ordenado, sendo essa a versão aprimorada do Bubble Sort.

Dessa forma, a sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado. Nessa situação, o vetor só é percorrido uma única vez, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes devido ao crescimento quadrático das operações de comparação e movimentação de dados.

2.2 Insertion Sort

O Insertion Sort (Ordenação por Inserção) consiste em inserir um novo elemento em um vetor já ordenado ou quase ordenado. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é inserida. Ele percorre o vetor a partir do segundo elemento (índice 1), selecionando-o como uma "chave", e compara essa chave com os elementos que estão nas posições anteriores (à sua esquerda). Caso os elementos anteriores sejam maiores que a chave, eles são deslocados uma posição para a direita para abrir espaço, e esse processo se repete até encontrar a posição correta, onde a chave é finalmente inserida.

Assim, sua complexidade de tempo é $O(n^2)$, exceto no caso em que o vetor já está ordenado (ou quase ordenado). Nessa situação, a chave é comparada apenas uma vez com o elemento imediatamente anterior e o loop interno é interrompido por um break, tornando a complexidade $O(n)$. É um algoritmo estável e in-place, mas seu comportamento degrada-se drasticamente em entradas grandes e aleatórias.

2.3 Quick Sort

O algoritmo de ordenação Quick Sort foi desenvolvido e publicado pela primeira vez em 1961, pelo cientista da computação britânico Charles Antony Richard Hoare (1934 - 2026). A técnica de ordenação foi concebida enquanto o cientista estudava na Universidade de Moscou e trabalhava no *National Physical Laboratory (NPL)*, do Reino Unido. Na ocasião, Hoare estava participando de um projeto que tinha como objetivo a tradução de textos em russo para inglês e, para isso, como os dicionários eram armazenados em fita magnética, necessitava ordenar as palavras dentro de uma sentença em ordem alfabética. Nesse contexto, surgiu a necessidade de desenvolver um algoritmo de ordenação que fosse assintoticamente menor que $O(n^2)$.

Sob essa ótica, o algoritmo é baseado no paradigma da divisão e conquista. Nesse sentido, a aplicação se consiste em ordenar o vetor em torno de um pivô escolhido, de modo que, os elementos à esquerda do pivô sejam menores que ele e os elementos à direita, consequentemente, sejam maiores. Dessa forma, o vetor é subdividido em duas partições, que terão o mesmo processo aplicado sobre elas, através de chamadas recursivas da função. Assim, as chamadas se encerram quando o caso base de um elemento na partição é atingido, uma vez que ele já estará ordenado.

Tais comportamentos destacados são os aspectos base do algoritmo quick sort, no entanto, cada aplicação dista da técnica utilizada para escolher o elemento pivô e para como o particionamento será realizado. Para o algoritmo implementado neste projeto, utilizou-se o método da mediana de 3 para se determinar o pivô. Ele se consiste em armazenar o primeiro, central e último elemento do vetor em variáveis auxiliares e retornar a mediana entre eles, nesse caso, o elemento do meio. Essa técnica, embora seja mais complexa do que aplicações comuns que utilizam o primeiro ou último elemento do array como pivôs, tem como benefícios a melhora do tempo de execução e a possibilidade da execução possuir uma complexidade de $O(n \log n)$ mesmo para vetores ordenados ou parcialmente ordenados - embora as métricas de análise dos vetores utilizadas para esse projeto evitem tais casos.

No caso da partição, existem três principais métodos comumente utilizados para o quick sort, o particionamento de Lomoto, Naive e Hoare, para este trabalho se utilizou o particionamento de Hoare. O método se consiste em na definição de dois ponteiros, nas duas extremidades do vetor e que, a cada interação, se movem em direção de seu encontro até que um elemento que necessita ser trocado de lugar seja encontrado. Considerando i o elemento na posição zero do vetor e j o elemento na posição final ambos são movidos até que i encontre um elemento maior que o pivô e, analogamente, j encontre um elemento menor que o pivô. Nesse caso, os valores encontrados trocam de posição e i e j voltam a percorrer o vetor até que eles se cruzem ou sobreponham. Destaca-se, que assim como a partição de Lomoto, o algoritmo de Hoare possui uma complexidade de tempo de $O(n)$ e de espaço de $O(1)$, contudo, ela é considerada mais rápida que o outro método, uma vez que, no geral, realiza menos trocas entre elementos. Em relação ao particionamento de Naive, o algoritmo se baseia na criação de um vetor auxiliar, ou seja, a complexidade de espaço é de $O(n)$, sendo significativamente menos eficiente que o método utilizado.

Dado as características da partição de Hoare, é importante destacar que o quick sort é um algoritmo *in-place*, ou seja, não utiliza nenhum vetor auxiliar para a ordenação, uma vez que as trocas são realizadas dentro do próprio vetor. Além disso, o algoritmo não é estável, uma vez que não preserva a ordem relativa entre elementos de mesmo valor. É também importante destacar que o quick sort apresenta um comportamento muito eficiente em termos de acesso à memória devido à forma como realiza o particionamento do vetor. Durante essa etapa, os elementos são percorridos sequencialmente por meio de índices que avançam a partir das extremidades do arranjo, favorecendo a localidade espacial dos dados. Como elementos próximos na memória tendem a ser acessados em instantes próximos, a hierarquia de memória do processador, especialmente os níveis de cache, é utilizada de maneira mais eficiente. Além disso, à medida que as partições são realizadas, o algoritmo passa a operar sobre subvetores progressivamente menores, os quais frequentemente passam a caber integralmente na memória cache. Dessa forma, reduz-se a quantidade de acessos à memória principal, resultando em uma diminuição da latência de acesso aos dados e contribuindo significativamente para o elevado desempenho prático do algoritmo.

Ademais, é possível analisar o melhor, pior e médio caso do quick sort. Nota-se que o melhor caso ocorre quando o pivô selecionado é exatamente o elemento central do vetor analisado.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Pela análise da recursão, através do Teorema Mestre, obtém-se a complexidade de espaço para o melhor caso como $O(n \log n)$. Nesse sentido, para o médio caso, considera-se que o array é dividido em duas partes de tamanho K , sendo assim:

$$T(n) = T(n - K) + T(n)$$

Após a resolução da recorrência, também nota-se que a complexidade é de $O(n \log n)$. Nesse sentido, evidencia-se que no melhor e médio caso, o quick sort utiliza em média $2 n \log n$ comparações entre elementos distintos do vetor para ordená-lo, e aproximadamente, um sexto desse número de trocas. Por fim, o pior caso ocorre quando o pivô escolhido é sempre o menor ou o maior elemento do subvetor analisado. Nessa situação, a cada etapa do algoritmo são geradas partições extremamente desbalanceadas, contendo 0 e $n - 1$ elementos, o que resulta em uma árvore de recursão linear e leva a uma complexidade temporal de $O(n^2)$:

$$T(n) = T(n - 1) + n$$

Para o pior caso, observa-se que o algoritmo necessita realizar, em média, $n^2/2$ comparações para ordenar o vetor, o que representa um aumento assintótico significativo em relação aos outros casos. Em relação à complexidade de espaço, cada chamada recursiva apresenta complexidade $O(1)$ - devido a característica *in-place* do algoritmo -, no entanto, múltiplas chamadas ocorrem simultaneamente, modificando o comportamento da complexidade espacial conforme o caso de execução. Para o médio e melhor caso, a árvore de recursão gerada é balanceada e possui profundidade de $O(\log n)$, o que acarreta em uma complexidade de espaço de $O(\log n)$. Em contrapartida, devido ao comportamento linear da árvore no pior caso, sua altura é de $O(n)$, acarretando em uma complexidade assintótica de $O(n)$. Assim, foi possível elaborar a seguinte tabela comparativa:

Varição	Complexidade de tempo	Complexidade de espaço
Melhor caso	$O(n \log n)$	$O(\log n)$
Caso médio	$O(n \log n)$	$O(\log n)$
Pior caso	$O(n^2)$	$O(n)$

Tabela 1: Complexidade de tempo e espaço do quick sort.

2.4 Heap Sort

explicação do Heap

2.5 Radix Sort

explicação do radix

3 Arquitetura do Sistema

Descrição: • das métricas coletadas; • das heurísticas; • das decisões implementadas - a parte da

bia

4 Metodologia Experimental

Descrever: • ambiente utilizado; • datasets; • tamanhos testados; • critérios de avaliação

5 Resultados

Apresentar tabelas, gráficos, métricas coletadas, com discussão crítica. Explicar: • decisões corretas; • decisões ruins; • comportamentos inesperados; • limitações observadas; • impacto das características da entrada. • impactos das entradas adversariais

5.1 Quick Sort

Com o intuito de validar a algoritmo quick sort implementado neste projeto, analisou-se as curvas do número de comparações entre elementos do vetor e número de trocas necessárias para atingir a ordenação - para valores com diferentes ordens de grandeza - quais foram estudadas teoricamente e possuem comportamento esperado definido. Dessa forma, nota-se que o número de comparações do quick sort para o melhor caso é na ordem de $2 n \log n$, e um sexto desse número para a quantidade de trocas.

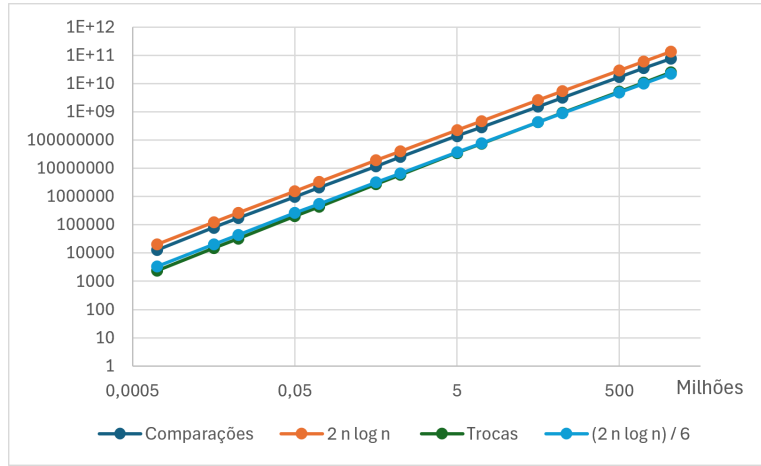


Figura 1: Comparativo entre o número de comparações e trocas e seu comportamento esperado para o melhor caso do quick sort. Escala dilog.

No gráfico é possível observar que o comportamento esperado é efetivo, uma vez que os dados coletados acompanham as linhas das funções que descrevem o comportamento previsto. Observa-se que o gráfico foi elaborado em escala log-log para a melhor visualização do comportamento para as diferentes ordens de grandeza.

Na implementação do quick sort desenvolvida, o pivô é escolhido por meio do método da mediana de três, utilizando os elementos inicial, central e final do subvetor. Essa estratégia reduz significativamente a ocorrência de partições desbalanceadas, especialmente em vetores já ordenados ou parcialmente ordenados. Ainda assim, o pior caso permanece teoricamente possível e ocorre quando a mediana desses três elementos produz repetidamente pivôs próximos aos extremos do subvetor, gerando partições de tamanhos aproximadamente 0 e $n - 1$. Nesse sentido, o método da mediana de três foi amplamente estudada por cientistas como David Musser, pesquisador conhecido por desenvolver o algoritmo Introsort, atualmente utilizado em diversas bibliotecas padrão da linguagem C++. Em seu trabalho, Musser demonstrou que, embora a estratégia da mediana de três reduza significativamente a probabilidade de partições desbalanceadas em entradas comuns, ela não elimina completamente o pior caso do Quick Sort. Sob essa ótica, existem famílias de sequências especialmente construídas, conhecidas como *Median-of-Three Killer Sequences*, capazes de induzir a escolha repetida de pivôs inadequados, resultando em partições altamente desbalanceadas e levando o algoritmo a uma complexidade temporal quadrática $O(n^2)$.

Algumas dessas sequências conhecidas são a *organ pipe* e intercalada. A sequência *organ pipe* se consiste em um array com o elemento central possuindo o maior valor do vetor e os elementos a esquerda e a direita dele decrescendo. Um exemplo de uma distribuição dessa forma, contínua, para $n = 10$ é: $[1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 4 \ 3 \ 2 \ 1]$. Em relação a distribuição intercalada, o vetor se consiste em valores intercalados entre uma sequência crescente e outra decrescente, dessa forma, para $n = 10$, uma possível

configuração seria $[1\ 10\ 2\ 9\ 3\ 8\ 4\ 7\ 5\ 6]$. No gráfico a seguir, é possível visualizar um comparativo entre os tempos de execução para vetores de distribuição aleatória e distribuição seguindo o padrão *organ pipe*:

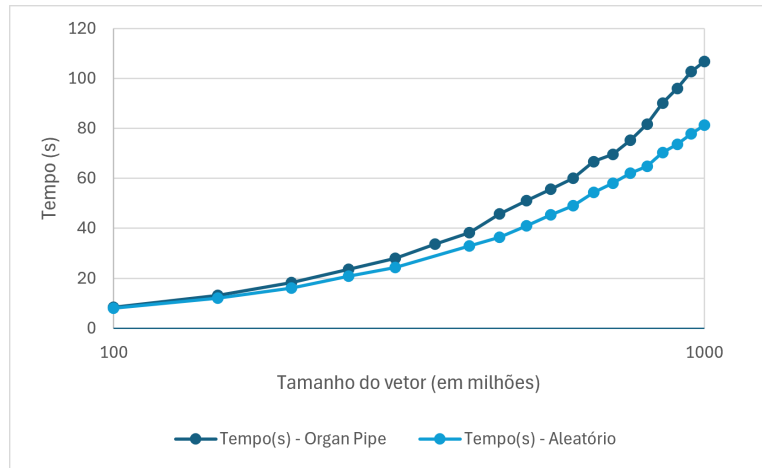


Figura 2: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Através dos dados, é possível observar que essa distribuição não estressou devidamente o algoritmo, uma vez que a curva de ambos apresentou uma complexidade temporal de $O(n \log n)$, embora a dsitribuição *organ pipe* tenha apresentado tempos de execução ligeiramente maiores. É válido destacar o comportamento da métrica que mede a profundidade máxima árvore de rescursão para ambos os casos.

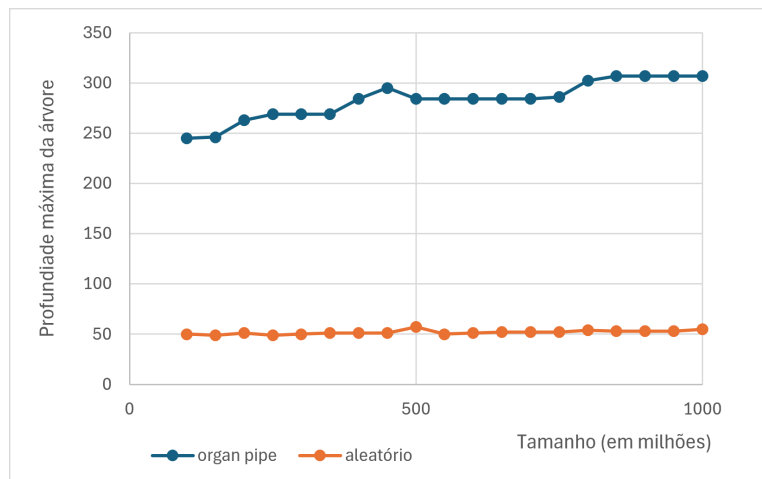


Figura 3: Comparatio de tempo de execução para a ordenação quick sort entre vetores de distribuição aleatória e quick sort.

Observa-se que a profundidade máxima para a distribuição *organ pipe* foi significativamente maior do que para o caso aleatório, indicando que a escolha do pivô para esses vetores não foi tão eficaz quanto o observado no melhor caso, o que acarretou na necessidade de um maior número de partições e, consequentemente, uma maior árvore recursiva. Assim, os arrays criandos seguindo o modelo *organ pipe* não estressaram o algoritmo implementado o suficiente para prejudicar a sua complexidade assintótica de tempo para $O(n^2)$, à medida que, como observado e analisado, a ditribuição se enquadra em um médio caso do quick sort.

Em relação aos vetores intercalados, o quick sort não conseguiu ordenar arrays com mais de 100 milhões de tamanho, uma vez que a péssima escolha dos pivôs para esse caso acarretaram em árvores

de recursão extremamente grandes - com profundidade máxima maior que 80.000 para vetores na ordem de 200 milhões de tamanho - o que ocasionou a interrupção prematura da ordenação devido ao esgotamento da pilha de chamadas. Dessa forma, observa-se que o pior caso esperado foi atingido, acarretando em uma execução na ordem de $O(n^2)$ e incapaz de ser finalizada para vetores grandes, como os analisados. Nesse sentido, evidencia-se também que para vetores menores aos analisados, de ordem de grandeza 10.000, a profundidade máxima da recursão atingiu valores muito maiores do que a ordenação de vetores aleatórios na casa de 100 milhões de tamanho - por exemplo, um vetor aleatório de tamanho 1 bilhão atingiu profundidade máxima de 55, enquanto um vetor intercalado de 100 mil, atingiu 507. Esses comportamentos destacam que a estratégia da mediana de três, embora eficiente em casos usuais, permanece suscetível a entradas adversárias capazes de produzir partições fortemente desbalanceadas.

6 Reflexão sobre Uso de IA

- O grupo deverá informar:
- quais ferramentas de IA foram utilizadas;
 - em quais etapas auxiliaram;
 - erros encontrados nas respostas geradas;
 - correções realizadas pelo grupo.

7 Conclusão

Resumo dos principais achados.

8 Referências Bibliográficas